



Επεξεργασία Φωνής και Φυσικής Γλώσσας

8ο εξάμηνο, Ακαδημαϊκή περίοδος 2024-2025

1η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

1	Εισαγωγή	2
2	Θεωρητική Ανάλυση	2
3	Βήματα Προπαρασκευής	4
4	Βήματα κυρίως μέρους	7
4.1	Προετοιμασία διαδικασίας αναγνώρισης φωνής για τη USC-TIMIT	7
4.2	Προετοιμασία γλωσσικού μοντέλου	10
4.3	Εξαγωγή Ακουστικών Χαρακτηριστικών	18
4.4	Εκπαίδευση ακουστικών μοντέλων και αποκωδικοποίηση προτάσεων	19

1 Εισαγωγή

Ο σκοπός της 1ης εργαστηριακής άσκησης είναι η υλοποίηση ενός συστήματος επεξεργασίας και αναγνώρισης φωνής, χρησιμοποιώντας το εργαλείο Kaldi. Θα παρουσιαστεί αρχικά το θεωρητικό υπόβαθρο που αφορά το σύστημα που θα υλοποιηθεί και στην συνέχεια θα αναλυθεί η ανάπτυξη ενός συστήματος το οποίο θα αφορά αναγνώριση φωνημάτων από ηχογραφήσεις της USC-TIMIT. Για την εκπαίδευση και εκτίμηση του συστήματος θα χρησιμοποιηθούν δεδομένα audio τα οποία παρουσιάζονται και αναλύονται στο προπαρασκευαστικό κομμάτι της αναφοράς. Ο σχεδιασμός του συστήματος στο κύριο κομμάτι της εργασίας θα αποτελείται από 4 μέρη:

- Εξαγωγή κατάλληλων ακουστικών χαρακτηριστικών από φωνητικά δεδομένα ((Mel-Frequency Cepstral Coefficient)
- Δημιουργία γλωσσικών μοντέλων από τα transcriptions του dataset
- Εκπαίδευση ακουστικών μοντέλων από τα ακουστικά χαρακτηριστικά που έχουν εξαχθεί
- Κατασκευή τελικού συστήματος αναγνώρισης φωνής

2 Θεωρητική Ανάλυση

Αρχικά, κρίνεται απαραίτητο να αναλυθούν κάποιες βασικές έννοιες που χρησιμοποιούνται στην συνέχεια:

1. Mel-Frequency Cepstral Coefficients (MFCCs)

Το mel-frequency cepstrum (MFC) αποτελεί έναν τρόπο να αναπαρασταθεί το φάσμα ισχύος βραχέος χρόνου ενός σήματος (short-term power spectrum). Οι παράγοντες που συνδυάζονται για την παραγωγή του MFC είναι οι MFCCs. Μέσω αυτών μπορεί να γίνει επεξεργασία της ανθρώπινης ομιλίας από μηχανές. Παρακάτω περιγράφουμε και σχολιάζουμε τον τρόπο εξαγωγής των παραγόντων αυτών από ένα σήμα ομιλίας:

Αρχικά, στο σήμα εφαρμόζεται ένα φίλτρο 1ου βαθμού, το οποίο ενισχύει τις υψηλές συχνότητες για καλύτερη ισορροπία του φάσματος ισχύος, αποφυγή προβλημάτων των αριθμητικών μεθόδων κατά την εφαρμογή του μετασχηματισμού Fourier και για την αύξηση του σηματοθορυβικού λόγου (signal to noise ratio - SNR):

$$y(t) = x(t) - \alpha x(t - 1)$$

όπου $x(t)$ είναι το αρχικό σήμα και τυπικές τιμές για την σταθερά α είναι 0.95 - 0.97.

Στην συνέχεια, με σκοπό την εξαγωγή των χαρακτηριστικών των συχνοτήτων σε συνάρτηση με τον χρόνο, το σήμα που έχει προκύψει χωρίζεται σε τμήματα διάρκειας 20 - 40 ms, τα οποία επικαλύπτονται κατά 50%(±10%). Στην συνέχεια, στα τμήματα εφαρμόζεται μια συνάρτηση παραθύρου Hamming (πολλαπλασιασμός στον χρόνο - συνέλιξη στον χώρο των συχνοτήτων):

$$w[n] = 0.54 - 0.46 \cos \frac{2\pi n}{N-1}, 0 \leq n \leq N-1$$

με N το μήκος του παραθύρου.

Το φάσμα ισχύος μπορεί να υπολογιστεί από τον μετασχηματισμό fourier βραχέος χρόνου (STFT) των τμημάτων x_i του σήματος x :

$$P = \frac{|FFT(x_i)|^2}{N}$$

Στο φάσμα ισχύος του σήματος εφαρμόζεται μια συστοιχία τριγωνικών φίλτρων στην κλίμακα Mel, η οποία μιμείται την μη γραμμικότητα της ανθρώπινης ακοής με ισχυρή διακριτική ικανότητα σε χαμηλές

συχνότητες που εξασθενεί στις υψηλότερες. Η μετατροπή από Hz σε κλίμακα Mel και αντίστροφα είναι η εξής:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

$$f = 700 (10^{m/2595} - 1)$$

Με τα παραπάνω βήματα έχουν παραχθεί οι παράγοντες των filter banks της κλίμακας Mel. Επειδή, όμως οι παράγοντες αυτοί εμφανίζουν έντονη συσχέτιση μεταξύ τους εφαρμόζεται ένας διακριτός μετασχηματισμός συνημιτόνου (DCT) με αποτέλεσμα να παραχθούν παράγοντες με μικρότερη συσχέτιση. Επιπλέον, μπορούν να διατηρηθούν για την συνέχεια της ανάλυσης ένας αριθμός από 2-13 αυτών των παραγόντων (οι πρώτοι), καθώς οι υπόλοιποι εκφράζουν έντονες αλλαγές των filter banks, λεπτομέρεια η οποία στην αυτόματη αναγνώριση φωνής δεν είναι ιδιαίτερα χρήσιμη. Οι παράγοντες που έχουν παραχθεί είναι οι MFCCs. Σημειώσεις για τους MFCCs:

- Η εξασθένιση των υψηλότερων MFCCs μπορεί να γίνει με ημιτονοειδές φιλτράρισμα στο πεδίο των συχνοτήτων (sinusoidal liftering).
- Για την εξισσορόπηση του φάσματος ισχύος και την ενίσχυση του σηματοθορυβικού λόγου, μπορεί να αφαιρεθεί από όλα τα τμήματα του σήματος ο μέσος όρος των παραγόντων.
- Στην παραπάνω ανάλυση παράχθηκαν παράγοντες filter banks και mfccs από ένα σήμα. Η μέθοδος υπολογισμού των filter banks πηγάζει από την φύση των σημάτων ομιλίας και τον τρόπο που τα αντιλαμβάνεται ο άνθρωπος. Το επιπλέον βήμα που οδηγεί στους παράγοντες MFCCs έχει νόημα για χρήση με GMMs και HMMs στην διαδικασία αυτόματης αναγνώρισης ομιλίας. Σε συστήματα βαθιάς μάθησης η συσχέτιση των παραγόντων δεν επηρεάζει σημαντικά την απόδοση στην αναγνώριση και μάλιστα ο γραμμικός μετασχηματισμός DCT απορρίπτει μη γραμμικά στοιχεία που ίσως είναι χρήσιμα. Σαν αποτέλεσμα, σε τέτοια συστήματα ίσως η απόδοση αυξηθεί χρησιμοποιώντας filter banks. Τέλος, εφόσον ακόμα και ο FFT είναι γραμμικός μετασχηματισμός θα μπορούσε να υποτεθεί ότι ίσως είναι καλύτερο ένα σύστημα deep learning για αυτόματη αναγνώριση ομιλίας να μαθαίνει κατευθείαν από το σύστημα. Αυτό, όμως έχει το αρνητικό ότι το μοντέλο θα έχει αυξημένη πολυπλοκότητα και θα χρειάζεται περισσότερες παραμέτρους στην εκπαίδευση, κάτι το οποίο εφόσον ο STFT χρησιμοποιείται σε σήματα που θεωρούνται στατικά δεν είναι σίγουρο ότι θα βελτιώνει την απόδοση.

2. Φωνητικό Μοντέλο (Acoustic Model)

Ένα φωνητικό μοντέλο είναι μοντέλο που εκπαιδεύεται με είσοδο ένα σήμα ήχου να αντιστοιχίζει τμήματά του σε πιθανότητες που αντιστοιχούν σε φωνήματα ή σε άλλα γλωσσικά στοιχεία.

3. Γλωσσικό Μοντέλο (Language Model)

Ένα γλωσσικό μοντέλο είναι εκπαιδευμένο να κατανέμει σε λέξεις την πιθανότητα να είναι η επόμενη λέξη σε μία πρόταση (με ευρύτερη έννοια) βασιζόμενο στις προηγούμενες λέξεις.

4. Αυτόματη Αναγνώριση Φωνής (Automatic Speech Recognition - ASR)

Το πρόβλημα της αυτόματης αναγνώρισης φωνής μπορεί να μοντελοποιηθεί ως η διαδικασία εύρεσης της λέξης $\hat{W}$ που μεγιστοποιεί την a posteriori πιθανότητα $P(W|X)$, δεδομένου του X , που είναι το διάλυμα χαρακτηριστικών (π.χ. παράγοντες MFCCs). Δηλαδή, εκφράζοντας σε μαθηματικό φορμαλισμό:

$$\hat{W} = \operatorname{argmax}_W P(W|X)$$

και από το θεώρημα Bayes:

$$\hat{W} = \operatorname{argmax}_W \frac{P(X|W)P(W)}{P(X)}$$

Σημειώνεται για τα παραπάνω ότι ο υπολογισμός της a posteriori πιθανότητας περιλαμβάνει 2 μέρη, πρώτον τον υπολογισμό της a priori πιθανότητας που αποτελεί το γλωσσικό μοντέλο και τον υπολογισμό της

πιθανότητας το W να έχει παράξει το διάνυσμα χαρακτηριστικών X , το οποίο αποτελεί το ακουστικό μοντέλο.

Μπορούμε να υλοποιήσουμε το φωνητικό μοντέλο που περιγράφεται πάνω ως ένα κρυφό μοντέλο Markov (HMM). Δεδομένου ενός HMM το οποίο έχει εκπαιδευτεί στα δεδομένα μπορούμε να υπολογίσουμε τις πιθανότητες του ακουστικού μοντέλου εκμεταλλευόμενοι τον ακολουθιακό χαρακτήρα των φωνημάτων ως εξής:

$$P(X|W) = \sum_S P(X|S)P(S|W)$$

όπου ουσιαστικά χρησιμοποιούμε όλες τις πιθανές χρονικές ακολουθίες των καταστάσεων και τις πιθανότητες τους, για να καταλήξουμε στο τελικό φωνητικό μοντέλο. Σημειώνεται ότι στην πραγματικότητα επειδή δεν είναι εφικτός ο υπολογισμός όλων των πιθανών καταστάσεων λόγω εκθετικής πολυπλοκότητας χρησιμοποιούνται τεχνικές δυναμικού προγραμματισμού για τον αποδοτικό υπολογισμό.

Στα παραπάνω, μπορούν να χρησιμοποιηθούν και μοντέλα GMM (Gaussian Mixture Models) για να μοντελοποιήσουν την πιθανότητα χαρακτηριστικών, δεδομένου ενός φωνήματος W στα HMMs ως εξής:

$$P(X|W) = \sum_i p_i N(\mu_i, \Sigma_i)$$

3 Βήματα Προπαρασκευής

Αρχικά εγκαταστήσαμε το εργαλείο kaldι στον υπολογιστή μας σύμφωνα με τις οδηγίες, καθώς και τα απαραίτητα δεδομένα με τις ηχογραφήσεις. Στη συνέχεια, για την κατασκευή του αρχικού σκελετού, δημιουργήσαμε ένα φάκελο **usc** μέσω στον φάκελο **/kaldi/egs**, ο οποίος θα αποτελέσει και τον κύριο φάκελο στον οποίο θα εργαζόμαστε για τα επόμενα μέρη του εργαστηρίου. Τα επόμενα βήματα παρουσιάζονται αναλυτικά μαζί με τα αντίστοιχα scripts παρακάτω:

1. Μέσα στον φάκελο **usc** δημιουργήσαμε το φάκελο **data** και τους υποφάκελους **data/train**, **data/dev**, **data/test**, μέσα στους οποίους θα δημιουργήσουμε αρχεία-δείκτες τα οποία θα περιγράφουν τα δεδομένα εκπαίδευσης, επαλήθευσης και αποτίμησης αντίστοιχα.
2. Μέσα σε κάθε φάκελο από τους παραπάνω δημιουργήσαμε ένα αρχείο **uttdids**, το οποίο περιέχει στην κάθε γραμμή ένα μοναδικό συμβολικό όνομα για κάθε πρόταση του συγκεκριμένου συνόλου δεδομένων (utterance_ids). Για την δημιουργία αυτών των αρχείων στην αντιγράφουμε τα περιεχόμενα των αρχείων στον φάκελο **filesets** στο αντίστοιχο subdirectory του **data** και τα μετανομάζουμε σε **uttdids**
3. Το επόμενο αρχείο που δημιουργήσαμε μέσα σε κάθε φάκελο είναι το **utt2spk**, το οποίο περιέχει σε κάθε γραμμή τον ομιλητή που αντιστοιχεί σε κάθε πρόταση. Το script για τη δημιουργία του είναι το εξής:

```
input_filename = "uttdids"
output_filename = "utt2spk"

with open(input_filename, "r") as infile:
    lines = infile.readlines()

with open(output_filename, "w") as outfile:
    for line in lines:
        line = line.strip()
        if line:
            prefix = line[:2]
            outfile.write(f"{line} {prefix}\n")

print(f"Processed data has been saved to {output_filename}")
```

Τα περιεχόμενα του αρχείου έχουν την εξής μορφή:

```
f1_003 f1
f1_004 f1
f1_005 f1
f1_007 f1
```

όπου το για παράδειγμα στην πρώτη γραμμή το f1_003 είναι το utterance id και το f1 ο αντίστοιχος ομιλητής.

4. Το επόμενο αρχείο που δημιουργήσαμε μέσα σε κάθε φάκελο είναι το wav.scp, το οποίο περιέχει τη θέση του αρχείου ήχου που αντιστοιχεί σε κάθε πρόταση. Το script που δημιουργήσαμε για την κατασκευή των αρχείων φαίνεται παρακάτω:

```
input_filename = "uttsids"
output_filename = "wav.scp"

# Read the input file
with open(input_filename, "r") as infile:
    lines = infile.readlines()

# Process each line
with open(output_filename, "w") as outfile:
    for line in lines:
        line = line.strip() # Remove any surrounding whitespace
        if line: # Ensure the line is not empty
            prefix = line[:2] # Extract first two letters
            path = f"/home/angeliki/Desktop/kaldi/egs/usc/wav/{line}.wav"
            outfile.write(f"{line} {path}\n")

print(f"Processed data has been saved to {output_filename}")
```

Τα περιεχόμενα του αρχείου έχουν την εξής μορφή:

```
f1_003 /home/angeliki/Desktop/kaldi/egs/usc/wav/f1_003.wav
f1_004 /home/angeliki/Desktop/kaldi/egs/usc/wav/f1_004.wav
f1_005 /home/angeliki/Desktop/kaldi/egs/usc/wav/f1_005.wav
```

5. Το τελευταίο αρχείο που δημιουργήσαμε μέσα σε κάθε φάκελο είναι το text, το οποίο περιέχει το κείμενο που αντιστοιχεί στην κάθε πρόταση. Για τη δημιουργία αυτών των αρχείων χρησιμοποιήσαμε πέρα από το **uttsids** και το αρχείο **transcriptions.txt** το οποίο την αντιστοίχιση ανάμεσα στα id των προτάσεων (χωρίς τον ομιλητή) και του περιεχομένου των προτάσεων. Το script που χρησιμοποιήθηκε είναι το εξής:

```
input_filename = "uttsids" # First input file with IDs
# Second input file with IDs and sentences
sentence_filename = "transcriptions.txt"
output_filename = "text"

# Read the sentence mapping from the second file
from collections import defaultdict
```

```

sentence_dict = defaultdict(list)
with open(sentence_filename, "r") as sentence_file:
    for line in sentence_file:
        parts = line.strip().split("\t", 1) # Split at tab character
        if len(parts) == 2:
            # Store sentences for all IDs
            sentence_dict[parts[0].strip()].append(parts[1].strip())

# Read the first input file
with open(input_filename, "r") as infile:
    lines = infile.readlines()

# Process each line
with open(output_filename, "w") as outfile:
    for line in lines:
        line = line.strip() # Remove any surrounding whitespace
        if line: # Ensure the line is not empty
            prefix = line.split("_")[0] # Extract prefix before underscore
            path = f"/home/angeliki/Desktop/kaldi/egs/usc/wav/{line}.wav"

            # Find sentences for the exact ID, or
            #if unavailable, try a more general match (e.g., f1_003 -> 003)
            sentences = sentence_dict.get(line,
            sentence_dict.get(line.split("_")[1], ["No sentence found"]))

            outfile.write(f"{line} {' | '.join(sentences)}\n")

print(f"Processed data has been saved to {output_filename}")

```

Τα περιεχόμενα του αρχείου έχουν την εξής μορφή:

```

f1_003 She is thinner than I am.
f1_004 Bright sunshine shimmers on the ocean.
f1_005 Nothing is as offensive as innocence.

```

Αφού είχαμε κατασκευάσει το αντίστοιχα αρχεία text στους φακέλους **data/train**, **data/dev**, **data/test**, έπρεπε για κάθε αρχείο text που δημιουργήσαμε να αντικαταστήσετε τις λέξεις που περιέχουν οι προτάσεις με τις αντίστοιχες αλληλουχίες φωνημάτων. Για το σκοπό αυτό χρησιμοποιήσαμε το αρχείο **lexicon.txt**, το οποίο αντιστοιχίζει κάθε λέξη της αγγλικής γλώσσας στην αλληλουχία φωνημάτων που της αντιστοιχεί. Επίσης, κάθε πρόταση πρέπει να ξεκινάει και να τελειώνει με το φώνημα της σιωπής **sil**. Το script που κατασκευάσαμε για να κάνουμε τα παραπάνω είναι το εξής:

```

import re

input_filename = "text" # dsFile with sentences
lexicon_filename = "lexicon.txt" # Lexicon mapping words to phonemes
output_filename = "text"

# Load lexicon into a dictionary
lexicon = {}

```

```

with open(lexicon_filename, "r") as lex_file:
    for line in lex_file:
        parts = line.strip().split()
        if len(parts) > 1:
            word = parts[0].lower()
            phonemes = " ".join(parts[1:])
            lexicon[word] = phonemes

# Function to convert text to phonemes
def text_to_phonemes(sentence):
    sentence = sentence.lower() # Convert to lowercase
    # Remove punctuation except single quotes
    sentence = re.sub(r"[^a-z0-9' ]", "", sentence)
    words = sentence.split()
    # Replace words with phonemes
    phoneme_sentence = [lexicon.get(word, word) for word in words]
    # Add silence phoneme at start & end
    return "sil " + " ".join(phoneme_sentence) + " sil"

# Process input file
with open(input_filename, "r") as infile, open(output_filename, "w") as outfile:
    for line in infile:
        line = line.strip()
        if not line:
            continue # Skip empty lines

        parts = line.split(" ", 1) # Split speaker ID and sentence
        if len(parts) == 2:
            speaker_id, sentence = parts
            phoneme_sentence = text_to_phonemes(sentence)
            outfile.write(f"{speaker_id} {phoneme_sentence}\n")
        else:
            outfile.write(f"{line}\n") # If no sentence, keep the line as is

print(f"Processed data has been saved to {output_filename}")

```

Το τελικό αρχείο text έχει την εξής μορφή:

```

f1_003 sil sh iy ih z th ih n er dh ae n ay ae m sil
f1_004 sil b r ay t s ah n sh ay n sh ih m er z aa n dh iy ow sh ah n sil
f1_005 sil n ah th ih ng ih z ae z ah f eh n s ih v ae z ih n ah s ah n s sil

```

4 Βήματα κυρίως μέρους

4.1 Προετοιμασία διαδικασίας αναγνώρισης φωνής για τη USC-TIMIT

1. Αρχικά χρησιμοποιήσαμε τις εντολές:

```

cp kaldi/egs/wsj/s5/path.sh kaldi/egs/usc/
cp kaldi/egs/wsj/s5/cmd.sh kaldi/egs/usc/

```

για να αντιγράψουμε τα δύο αυτά αρχεία στον βασικό μας φάκελο.

Στη συνέχεια, στο αρχείο path.sh θέτουμε τη μεταβλητή **KALDI_ROOT** να δείχνει στο directory που βρίσκεται ο κύριος φάκελος της εγκατάστασης του Kaldi. Επίσης, στο cmd.sh αλλάζουμε τις τιμές

των μεταβλητών **train_cmd**, **decode_cmd** και **cuda_cmd** σε run.pl. Οι αλλαγές αυτές φαίνονται παρακάτω:

```
export KALDI_ROOT=/home/angeliki/Desktop/kaldi
```

```
export train_cmd=run.pl
export decode_cmd=run.pl
# the use of cuda_cmd is deprecated, used only in 'nnet1',
export cuda_cmd=run.pl
```

2. Χρησιμοποιούμε το παρακάτω script για να δημιουργήσουμε soft links μέσα στο φάκελο /kaldi/egs/usc, με ονόματα **steps** και **utils** τα οποία θα δείχνουν στους αντίστοιχους φακέλους του kaldi/egs/ws/.

```
#!/bin/bash

source path.sh

# Define the target directories
TARGET_STEPS_DIR=~/Desktop/kaldi/egs/ws/s5/steps
TARGET_UTILS_DIR=~/Desktop/kaldi/egs/ws/s5/utils

# Define the symlink names
STEPS_LINK_NAME=steps
UTILS_LINK_NAME=utils

# Check if the symlink already exists
if [ -L "$STEPS_LINK_NAME" ]; then
    echo "Symbolic link '$STEPS_LINK_NAME' already exists."
else
    # Check for the target directory
    if [ ! -d "$TARGET_STEPS_DIR" ]; then
        echo "Error: Target directory '$TARGET_STEPS_DIR' does not exist."
        exit 1
    fi
    #Symbolic link for steps
    ln -s "$TARGET_STEPS_DIR" "$STEPS_LINK_NAME"
    echo "Symbolic link '$STEPS_LINK_NAME' created -> $TARGET_STEPS_DIR"
fi

# Check if the symlink already exists
if [ -L "$UTILS_LINK_NAME" ]; then
    echo "Symbolic link '$UTILS_LINK_NAME' already exists."
else
    # Check for the target directory
    if [ ! -d "$TARGET_UTILS_DIR" ]; then
        echo "Error: Target directory '$TARGET_UTILS_DIR' does not exist."
        exit 1
    fi
    #Symbolic link for utils
    ln -s "$TARGET_UTILS_DIR" "$UTILS_LINK_NAME"
    echo "Symbolic link '$UTILS_LINK_NAME' created -> $TARGET_UTILS_DIR"
fi
```


Σημειώνουμε ότι από εδώ και πέρα το αρχείο **path.sh** θα το κάνουμε source στην αρχή κάθε bash script, ώστε να έχουμε διαθέσιμες όλες τις εντολές του Kaldi.

3. Στη συνέχεια δημιουργούμε τον φάκελο **local** με τη χρήση της εντολής

```
mkdir local
```

Αφού έχουμε δημιουργήσει τον φάκελο, δημιουργούμε μέσα σε αυτόν ένα soft link που να δείχνει στο αρχείο **score_kaldi.sh** που βρίσκεται μέσα στο **steps**. Για το παραπάνω χρησιμοποιήσαμε το εξής script:

```
#!/bin/bash
source path.sh

#Define path to the original file and the destination folder
SOURCE_SCORE_SCRIPT=~/Desktop/kaldi/egs/wsj/s5/steps/score_kaldi.sh
DEST_DIR=local
LINKED_SCORE_SCRIPT=score_kaldi.sh

# Create the destination folder if it doesn't exist
if [ ! -d "$DEST_DIR" ]; then
    mkdir -p "$DEST_DIR"
    echo "Destination folder '$DEST_DIR' created."
fi

# Check if the source file exists
if [ ! -f "$SOURCE_SCORE_SCRIPT" ]; then
    echo "Error: Source file '$SOURCE_SCORE_SCRIPT' does not exist."
    exit 1
fi

# Create the symbolic link
ln -s "$SOURCE_SCORE_SCRIPT" "$DEST_DIR/$LINKED_SCORE_SCRIPT"

# Verify and print success message
if [ -L "$DEST_DIR/$LINKED_SCORE_SCRIPT" ]; then
    echo "Symbolic link '$LINKED_SCORE_SCRIPT' created in '$DEST_DIR' -> $SOURCE_SCORE_SCRIPT"
else
    echo "Error: Failed to create symbolic link."
    exit 1
fi
```

4. Δημιουργούμε τον φάκελο **conf**:

```
mkdir conf
```

και στη συνέχεια αντιγράφουμε σε αυτόν το αρχείο **mfcc.conf**, το οποίο μας δόθηκε.

5. Τέλος, δημιουργούμε μέσα στον φάκελο **data** τους εξής φακέλους: **data/lang**, **data/local/dict**, **data/local/lm_tmp**, **data/local/nist_lm**. Το script που χρησιμοποιήσαμε ήταν το εξής:

```
#!/bin/bash
source path.sh

# List of folders to create
FOLDERS_TO_CREATE=("data/lang" "data/local/dict" "data/local/lm_tmp"
"data/local/nist_lm")

# Loop through each folder and create if it doesn't exist
for folder in "${FOLDERS_TO_CREATE[@]"; do
    if [ ! -d "$folder" ]; then
        mkdir -p "$folder"
        echo "Created directory: $folder"
    else
        echo "Directory already exists: $folder"
    fi
done
```

4.2 Προετοιμασία γλωσσικού μοντέλου

1. Για το βήμα αυτό θα δουλέψουμε στον φάκελο `kaldi/egs/usc/data/local`, οπου θα δημιουργήσουμε κάποια αρχεία. Αρχικά, δημιουργούμε τα αρχεία `silence_phones.txt` και `optional_silence.txt`:

```
touch silence_phones.txt
touch optional_silence.txt
```

Τα αρχεία αυτά θα περιέχουν μόνο το φώνημα της σιωπής (sil). Στη συνέχεια δημιουργούμε το αρχείο `nonsilence_phones.txt`:

```
touch nonsilence_phones.txt
```

Το αρχείο αυτό πρέπει να περιέχει όλα τα υπόλοιπα φωνήματα εκτός από το sil (1 σε κάθε γραμμή και sorted). Για να το επιτύχουμε αυτό χρησιμοποιούμε το παρακάτω script το οποίο κάνει iterate σε όλες τις γραμμές του λεξικού και βρίσκει τα φωνήματα.

```
import re

# Define input and output file paths
input_file = "lexicon.txt"
output_file = "nonsilence_phones.txt"

# Read the lexicon file and extract phonemes
phoneme_set = set()
with open(input_file, "r", encoding="utf-8") as f:
    for line in f:
        parts = line.strip().split("\t")
        if len(parts) > 1: # Ensure there's a phonetic transcription
            phonemes = re.split(r'\s+', parts[1]) # Split by spaces
            phoneme_set.update(phonemes) # Add phonemes to the set

# Sort the phonemes alphabetically
sorted_phonemes = sorted(phoneme_set)
```

```
# Write the sorted phonemes to a new file
with open(output_file, "w", encoding="utf-8") as f:
    for phoneme in sorted_phonemes:
        f.write(phoneme + "\n")

print(f"Sorted phonemes written to {output_file}")
```

Το επόμενο αρχείο που θα δημιουργήσουμε θα είναι το **lexicon.txt**, το οποίο θα περιέχει όλα τα φωνήματα (συμπεριλαμβανομένου και του sil). Πιο συγκεκριμένα, κάθε γραμμή θα περιέχει ένα φώνημα, έπειτα κενό, και μετά πάλι το ίδιο φώνημα. Για να πετύχουμε το παραπάνω θα χρησιμοποιήσουμε το αρχείο που δημιουργήσαμε στο προηγούμενο βήμα, αφού έχουμε συμπεριλάβει και το φώνημα sil. Το script που χρησιμοποιήσαμε είναι το εξής:

```
input_file = "nonsilence_phones.txt"
output_file = "lexicon.txt"

with open(input_file, "r") as infile, open(output_file, "w") as outfile:
    for line in infile:
        phoneme = line.strip() # Remove any trailing newline/spaces
        if phoneme:           # Skip empty lines if any
            outfile.write(f"{phoneme} {phoneme}\n")

print(f"Created {output_file} with each phoneme paired with itself.")
```

Έπειτα, για καθένα από τα subdirectories **data/train**, **data/dev**, **data/test** δημιουργούμε το αρχείο **lm_{train, dev, test}.text** αντίστοιχα. Το κάθε αρχείο θα είναι στην ουσία το αντίστοιχο αρχείο txt, το οποίο θα έχει τις ειδικές μονάδες <s> και </s> στην αρχή και στο τέλος της κάθε πρότασης αντίστοιχα. Το script που εκτελεί την συγκεκριμένη διαδικασία είναι το εξής:

```
input_file = "text"
output_file = "output.txt"

with open(input_file, "r", encoding="utf-8") as infile,
    open(output_file, "w", encoding="utf-8") as outfile:
    for line in infile:
        line = line.strip()
        if not line:
            continue # Skip any empty lines

        tokens = line.split()
        sentence_id = tokens[0] # e.g., f1_003
        sentence_tokens = tokens[1:] # e.g., ['sil', 'sh', 'iy', ...]

        # Construct the new sentence with <s> and </s>
        new_sentence = f"{sentence_id} <s> {' '.join(sentence_tokens)} </s>"

        outfile.write(new_sentence + "\n")
```

```
print(f"Created '{output_file}' with <s> and </s> around each sentence.")
```

Το τελικό αρχείο έχει την εξής μορφή:

```
f1_002 <s> sil jh ey n m ey er n m ao r m ah n iy b ay w er k ih ng hh aa r d sil </s>
f1_014 <s> sil ah r ow l ah v w ay er l ey n ih r dh iy w ao l sil </s>
f1_027 <s> sil hh eh l p s eh l ah b r ey t y ao r b r ah dh er z s ah k s eh s sil </s>
```

Τέλος, δημιουργούμε το αρχείο `extra_questions.txt`, το οποίο θα είναι κενό.

```
touch extra_questions.txt
```

2. Στο βήμα αυτό θα μπούμε στον φάκελο `data/local/lm_tmp` και θα δημιουργήσουμε το ενδιαμέσο στάδιο του γλωσσικού μοντέλου για unigram και bigram. Για να το κάνουμε αυτό θα χρησιμοποιήσουμε την εντολή `build-lm.sh`, όπως φαίνεται στο παρακάτω script:

```
# Needed for KALDI_ROOT
source path.sh

exportIRSTLM=$KALDI_ROOT/tools/irstlm/
exportPATH=${PATH}:${IRSTLM}/bin

DATA_PATH=~ /Desktop/kaldi/egs/usc/data/local/dict
OUTPUT_PATH=./lm_tmp

build-lm.sh -i "$DATA_PATH/lm_train.text" -n 1 -o lm_train1.ilgm.gz
build-lm.sh -i "$DATA_PATH/lm_train.text" -n 2 -o lm_train2.ilgm.gz
build-lm.sh -i "$DATA_PATH/lm_test.text" -n 1 -o lm_test1.ilgm.gz
build-lm.sh -i "$DATA_PATH/lm_test.text" -n 2 -o lm_test2.ilgm.gz
build-lm.sh -i "$DATA_PATH/lm_dev.text" -n 1 -o lm_dev1.ilgm.gz
build-lm.sh -i "$DATA_PATH/lm_dev.text" -n 2 -o lm_dev2.ilgm.gz
```

Με την εντολή αυτή κάναμε στην ουσία build το κάθε γλωσσικό μοντέλο (για train , test και dev) και δημιουργήθηκαν τα αντίστοιχα αρχεία στον φάκελο `data/local/lm_tmp`.

3. Στη συνέχεια, μεταβαίνουμε στον φάκελο `data/local/nist_lm`, όπου θα αποθηκευτεί το compiled γλωσσικό μοντέλο σε μορφή ARPA. Για να κάνουμε compile τα μοντέλα χρησιμοποιούμε την εντολή `compile-lm`.

```
# Load Kaldi environment
source ./path.sh # Sets up KALDI_ROOT

# Set and export IRSTLM path
exportIRSTLM=$KALDI_ROOT/tools/irstlm
exportPATH=$PATH:$IRSTLM/bin

# Compile UNIGRAM models (train1, test1, dev1)
compile-lm lm_train1.ilgm.gz --text=yes lm_train1.lm
compile-lm lm_train1.ilgm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_ug_train.arpa.gz
```

```

compile-lm lm_test1.ilm.gz --text=yes lm_test1.lm
compile-lm lm_test1.ilm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_ug_test.arpa.gz

compile-lm lm_dev1.ilm.gz --text=yes lm_dev1.lm
compile-lm lm_dev1.ilm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_ug_dev.arpa.gz

# Compile BIGRAM models (train2, test2, dev2)
compile-lm lm_train2.ilm.gz --text=yes lm_train2.lm
compile-lm lm_train2.ilm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_bg_train.arpa.gz

compile-lm lm_test2.ilm.gz --text=yes lm_test2.lm
compile-lm lm_test2.ilm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_bg_test.arpa.gz

compile-lm lm_dev2.ilm.gz --text=yes lm_dev2.lm
compile-lm lm_dev2.ilm.gz --text=yes /dev/stdout | grep -v unk |
gzip > lm_phone_bg_dev.arpa.gz

```

Όπως φαίνεται και παραπάνω, η εντολή κατά την εκτέλεση της αναμένει input από τον χρήστη, το οποίο έχουμε ενσωματώσει στο παραπάνω script. Μετά την εκτέλεση του script δημιουργούνται στον αντίστοιχο φάκελο τα compiled γλωσσικά μοντέλα σε μορφή arpa.

4. Στη συνέχεια δημιουργούμε το FST του λεξικού της γλώσσας χρησιμοποιώντας την εντολή **prepare_lang.sh**, όπως φαίνεται στο παρακάτω script:

```

# Source Kaldi environment and command configuration
source ./path.sh      # Initializes KALDI_ROOT and related paths
source ./cmd.sh        # Loads command setup (e.g., for queueing)

# Set up IRSTLM tool path and update system PATH
export IRSTLM="$KALDI_ROOT/tools/irstlm"
export PATH="$PATH:$IRSTLM/bin"

# Prepare language data using dictionary and placeholder for OOV
./utils/prepare_lang.sh data/local/dict "<ooov>" data/local/lm_tmp data/lang

```

5. Στο επόμενο βήμα πρέπει να ταξινομήσουμε τα αρχεία **wav.scp**, **text** και **utt2spk** στους φακέλους **data/train**, **data/dev** και **data/test**. Για να το επιτύχουμε αυτό θα χρησιμοποιήσουμε την εντολή **sort**, την οποία τρέξαμε αρχικά στο terminal, αλλά είδαμε ότι δεν αποθηκεύει το sorting. Γι' αυτό το λόγο την τρέξαμε με το παρακάτω script:

```

#!/bin/bash

# Load Kaldi path variables

```

```

source path.sh

# Set base data directory
DATA_DIR=~/Desktop/kaldi/egs/slp_lab/data"

# Process folders: train, dev, test
for split in train dev test; do
    SPLIT_DIR="$DATA_DIR/$split"

    if [ -d "$SPLIT_DIR" ]; then
        echo "Processing: $SPLIT_DIR"

        for file in wav.scp text utt2spk; do
            FILE_PATH="$SPLIT_DIR/$file"
            if [ -f "$FILE_PATH" ]; then
                sort "$FILE_PATH" -o "$FILE_PATH"
            else
                echo " Warning: $file not found in $split"
            fi
        done

    else
        echo "Skipping missing directory: $SPLIT_DIR"
    fi
done

echo "File sorting finished."

```

6. Αφού έχουμε κάνει το sorting θα εκτελέσουμε το script `utils/utt2spk_to_spk2utt.pl` ώστε να δημιουργήσετε το αρχείο **spk2utt**, το οποίο είναι το ίδιο με το `utt2spk`, αλλά με τις στήλες ανάποδα. Το script που χρησιμοποιήσαμε είναι το εξής:

```

#!/bin/bash
# Load Kaldi environment variables
source path.sh
# List of dataset splits
SETS=("train" "test" "dev")

# Generate spk2utt from utt2spk for each dataset
for split in "${SETS[@]}; do
    INPUT_FILE="data/$split/utt2spk"
    OUTPUT_FILE="data/$split/spk2utt"

    if [ -f "$INPUT_FILE" ]; then
        utils/utt2spk_to_spk2utt.pl < "$INPUT_FILE" > "$OUTPUT_FILE"
        echo "Created spk2utt for: $split"
    else
        echo "Missing utt2spk in $split | skipping"
    fi
done

```

Ουσιαστικά με το παραπάνω αρχείο κάνουμε iterate σε όλα τα directories `data/train`, `data/dev` και `data/test` και με την χρήση της εντολής που αναφέραμε παραπάνω δημιουργούμε τα ζητούμενα αρχεία.

7. Τέλος, θα πρέπει να δημιουργήσουμε το FST της γραμματικής (`G.fst`). Για να το κάνουμε αυτό μεταβήκαμε στον φάκελο `kaldi/egs/timit/s5/local` και αντιγράψαμε το αρχείο `timit-format_data.sh` στον κύριο φάκελό μας. Στη συνέχεια το τροποποιήσαμε ώστε να διαγράψουμε τις περιττές εντολές οι οποίες αντιστοιχούσαν σε βήματα τα οποία είχαμε ήδη πραγματοποιήσει στα προηγούμενα ερωτήματα. Παράλληλα δώσαμε τα σωστά ορίσματα, δηλαδή τα ονόματα των μοντέλων που κάναμε compile σε προηγούμενο ερώτημα.

```
#!/bin/bash

# Copyright 2013 (Author: Daniel Povey)
# Apache 2.0

# This script takes data prepared in a corpus-dependent way
# in data/local/, and converts it into the "canonical" form,
# in various subdirectories of data/, e.g. data/lang, data/train, etc.

. ./path.sh || exit 1;

echo "Preparing train, dev and test data"
srcdir=data/local/dict
lmdir=data/local/nist_lm
tmpdir=data/local/lm_tmp
lexicon=data/local/dict/lexicon.txt
mkdir -p $tmpdir

# Next, for each type of language model, create the corresponding FST
# and the corresponding lang_test_* directory.

echo "Preparing language models for test"

for x in train dev test; do
  test=data/lang_test
  mkdir -p $test
  cp -r data/lang/* $test

  gunzip -c $lmdir/lm_phone_bg_${x}.arpa.gz \
    | arpa2fst --disambig-symbol=#0 \
      --read-symbol-table=$test/words.txt - data/lang_test/G_${x}_bg.fst
  # The output is like:
  # 9.14233e-05 -0.259833
  # we do expect the first of these 2 numbers to be close to zero (the second
  # is nonzero because the backoff weights make the states sum to >1).
  # Because of the <s> fiasco for these particular LMs, the first number is not
  # as close to zero as it could be.

done
```

```

for x in train dev test; do
    test=data/lang_test

    gunzip -c $lmdir/lm_phone_ug_${x}.arpa.gz \
    | arpa2fst --disambig-symbol=#0 \
    --read-symbol-table=$test/words.txt - data/lang_test/G_${x}_ug.fst
    # The output is like:
    # 9.14233e-05 -0.259833
    # we do expect the first of these 2 numbers to be close to zero (the second
    # is nonzero because the backoff weights make the states sum to >1).
    # Because of the <s> fiasco for these particular LMs, the first number is not
    # as close to zero as it could be.

done

#utils/validate_lang.pl data/lang_test || exit 1

echo "Succeeded in formatting data."

```

Ερώτημα 1

Αφού έχουμε εκτελέσει όλα τα βήματα για την προετοιμασία του μοντέλου θα υπολογίσουμε το perplexity στο validation και στο test set. Το perplexity είναι στην ουσία ένα μέτρο αξιολόγησης της απόδοσης ενός γλωσσικού μοντέλου. Αντικατοπτρίζει το πόσο καλά προβλέπει το μοντέλο μια ακολουθία λέξεων. Συγκεκριμένα, όσο χαμηλότερη είναι η τιμή του perplexity, τόσο καλύτερη είναι η πρόβλεψη του μοντέλου, δηλαδή το μοντέλο είναι λιγότερο "απορημένο" με τα δεδομένα που βλέπει. Με άλλα λόγια, το perplexity είναι ουσιαστικά ο μέσος αριθμός επιλογών που πιστεύει το μοντέλο ότι έχει. Μαθηματικά, είναι εκθετική συνάρτηση της μέσης αρνητικής λογαριθμικής πιθανότητας των λέξεων σε ένα σύνολο αξιολόγησης.

$$\text{Perplexity}(W) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i)}$$

Το script με το οποίο υπολογίζουμε το perplexity είναι το εξής

```

. ./path.sh || exit 1;
. ./cmd.sh || exit 1;

cd "data/local/lm_tmp"
exportIRSTLM=$KALDI_ROOT/tools/irstlm/
exportPATH=${PATH}:${IRSTLM}/bin

echo "-----###-----"
echo "Perplexity for evaluation data unigram model"
echo "-----###-----"
prune-lm --threshold=1e-6,1e-6 lm_dev1.ilbm.gz lm_dev1.plm
compile-lm lm_dev1.lm --eval=../dict/lm_dev.text --dub=10000000

echo "-----###-----"
echo "Perplexity for evaluation data bigram model"
echo "-----###-----"
prune-lm --threshold=1e-6,1e-6 lm_dev2.ilbm.gz lm_dev2.plm

```



```

compile-lm lm_dev2.lm --eval=../dict/lm_dev.text --dub=10000000

echo "-----###-----"
echo "Perplexity for test data unigram model"
echo "-----###-----"
prune-lm --threshold=1e-6,1e-6 lm_test1.ilbm.gz lm_test1.plm
compile-lm lm_test1.lm --eval=../dict/lm_test.text --dub=10000000

echo "-----###-----"
echo "Perplexity for test data bigram model"
echo "-----###-----"
prune-lm --threshold=1e-6,1e-6 lm_test2.ilbm.gz lm_test2.plm
compile-lm lm_test2.lm --eval=../dict/lm_test.text --dub=10000000

```

Τα αποτελέσματα φαίνονται στις παρακάτω εικόνες:

```

-----###-----
Perplexity for evaluation data unigram model
-----###-----
OOV code is 190
OOV code is 190
pruning LM with thresholds:
1e-06
DEBUG_LEVEL:0/1 Everything OK
inpfile: lm_dev1.lm
outfile: lm_dev1.lm.blm
evalfile: ../dict/lm_dev.text
loading up to the LM level 1000 (if any)
dub: 10000000
OOV code is 190
OOV code is 190
Start Eval
OOV code: 190
%% Nw=5078 PP=39.83 PPwp=0.00 Nbo=0 Noov=0 OOV=0.00%

```

Figure 1: Perplexity for evaluation data for Unigram

```

-----###-----
Perplexity for evaluation data bigram model
-----###-----
OOV code is 190
OOV code is 190
pruning LM with thresholds:
1e-06
DEBUG_LEVEL:0/1 Everything OK
inpfile: lm_dev2.lm
outfile: lm_dev2.lm.blm
evalfile: ../dict/lm_dev.text
loading up to the LM level 1000 (if any)
dub: 10000000
OOV code is 190
OOV code is 190
Start Eval
OOV code: 190
%% Nw=5078 PP=14.97 PPwp=0.00 Nbo=147 Noov=0 OOV=0.00%

```

Figure 2: Perplexity for evaluation data for Bigram

```

-----###-----
Perplexity for test data unigram model
-----###-----
OOV code is 410
OOV code is 410
pruning LM with thresholds:
1e-06
DEBUG_LEVEL:0/1 Everything OK
inpfile: lm_test1.lm
outfile: lm_test1.lm.blm
evalfile: ../dict/lm_test.text
loading up to the LM level 1000 (if any)
dub: 10000000
OOV code is 410
OOV code is 410
Start Eval
OOV code: 410
%% Nw=13130 PP=40.22 PPwp=0.00 Nbo=0 Noov=0 OOV=0.00%

```

Figure 3: Perplexity for test data for Unigram

```

-----###-----
Perplexity for test data bigram model
-----###-----
OOV code is 410
OOV code is 410
pruning LM with thresholds:
1e-06
DEBUG_LEVEL:0/1 Everything OK
inpfile: lm_test2.lm
outfile: lm_test2.lm.blm
evalfile: ../dict/lm_test.text
loading up to the LM level 1000 (if any)
dub: 10000000
OOV code is 410
OOV code is 410
Start Eval

```

Figure 4: Perplexity for test data for Bigram

Οι τιμές perplexity που προέκυψαν από την αξιολόγηση των γλωσσικών μοντέλων παρουσιάζουν σημαντικές διαφορές μεταξύ των μοντέλων unigram και bigram. Συγκεκριμένα, στο validation set (dev), το μοντέλο unigram είχε perplexity ίσο με **39.83**, ενώ το αντίστοιχο bigram μοντέλο είχε τιμή **14.97**. Αντίστοιχα, στο test set, το perplexity ήταν **40.22** για το unigram και επίσης **14.97** για το bigram μοντέλο.

Η σημαντική μείωση του perplexity στο bigram μοντέλο οφείλεται στο ότι αυτό λαμβάνει υπόψη τη συμφραζόμενη πληροφορία, δηλαδή τη συσχέτιση μεταξύ διαδοχικών λέξεων, γεγονός που το καθιστά πιο «έξυπνο» στην πρόβλεψη της επόμενης λέξης.

Παρατηρείται επίσης ότι οι τιμές perplexity είναι παρόμοιες ανάμεσα στα validation και test sets, κάτι που υποδηλώνει ότι τα μοντέλα γενικεύουν καλά και δεν παρουσιάζουν φαινόμενα υπερπροσαρμογής (overfitting). Επιπλέον, το ποσοστό των λέξεων εκτός λεξιλογίου (OOV) ήταν μηδενικό σε όλα τα σετ, γεγονός που επιβεβαιώνει την πληρότητα του λεξικού και την ορθότητα της προεπεξεργασίας των δεδομένων. Συνολικά, το bigram μοντέλο επιδεικνύει σαφώς καλύτερη απόδοση ως προς την ικανότητά του να προβλέπει τις λέξεις στις ακολουθίες, όπως αποτυπώνεται από τη σημαντικά χαμηλότερη τιμή perplexity.

4.3 Εξαγωγή Ακουστικών Χαρακτηριστικών

Για την εξαγωγή των ακουστικών χαρακτηριστικών χρησιμοποιούμε το παρακάτω bash script:

```
. ./path.sh # Needed for KALDI_ROOT
. ./cmd.sh
#=====Make the mfcc=====
#Make MFCC features. mfcc_i where i = {train,test,dev} should be
#some place with a largish disk where you want to store
#MFCC features. nj number of parallel jobs - 1 is perfect for such a small data set
for x in train test dev; do
    steps/make_mfcc.sh --mfcc-config conf/mfcc.conf --cmd "$train_cmd" --nj 4 \
    data/$x exp/make_mfcc/$x mfcc_${x} || exit 1;
    steps/compute_cmvn_stats.sh data/$x exp/make_mfcc/$x mfcc || exit 1;
done
```

Εξάγουμε τα χαρακτηριστικά για τα set train, test και validation.

Ερώτημα 2

Στο παραπάνω script κάνουμε Cepstral Mean and Variable Normalization. Το script αυτό κανονικοποιεί τα χαρακτηριστικά, ώστε να έχουν μηδενική μέση τιμή και μοναδιαία μεταβλητότητα. Η κανονικοποίηση αυτή είναι σημαντική, καθώς μειώνει την μεταβλητότητα σε φωνητικά χαρακτηριστικά, βελτιώνει την ευρωστέια του μοντέλου εξασφαλίζοντας ότι όλα τα χαρακτηριστικά έχουν την ίδια κλίμακα και κατά την διάρκεια της εκπαίδευσης βοηθάει στην σύγκλιση.

Το αποτέλεσμα του script είναι το αρχείο data/set/cmvn.scp, όπου set train, test ή dev, το οποίο περιέχει πληροφορία για το byte offset που βρίσκονται τα δεδομένα του cmvn για κάθε speaker.

Στην συνέχεια με το παρακάτω script μπορούμε να βρούμε πληροφορίες για τις διαστάσεις των χαρακτηριστικών και τον αριθμό των ακουστικών frames κάθε πρότασης.

Σε διαδικασία αναγνώρισης φωνής θέλουμε συχνά να απαλλαγούμε από ανεπιθύμητο θόρυβο που μπορεί να προκύψει από το περιβάλλον. Ως γνωστόν η έξοδος ενός φίλτρου για ένα σήμα x είναι:

$$y[n] = x[n] * h[n]$$

Περνώντας στο πεδίο της συχνότητας:

$$Y[f] = H[f] \cdot X[f]$$

Εφαρμόζουμε λογάριθμο και έχουμε στο πεδίο του cepstrum:

$$Y[q] = X[q] + H[q]$$

Παρατηρούμε ότι τελικά οποιοσδήποτε θόρυβος υπήρχε στο αρχικό σήμα έχει μετατραπεί σε άθροισμα στο πεδίο του cepstrum.

Ο παραπάνω τύπος έχει την εξής μορφή για κάθε πλαίσιο:

$$Y_i[q] = X_i[q] + H[q]$$

Μπορούμε τώρα να υπολογίσουμε τον μέσο όρο των πλαισίων και την διαφορά του ενός πλαισίου από τον μέσο όρο, οποίος θα είναι η διαφορά του X στο παράθυρο από τον μέσο όρο όλων των παραθύρων. Προκύπτει, λοιπόν, κανονικοποίηση μέσης τιμής η οποία φροντίζει να αφαιρεθούν dc offsets. Τέλος διαιρώντας με την διασπορά μπορούμε να αποφύγουμε προβλήματα που έχουν πιθανόν προκύψει λόγω του scaling υπολογίζοντας το τελικό CMVN.

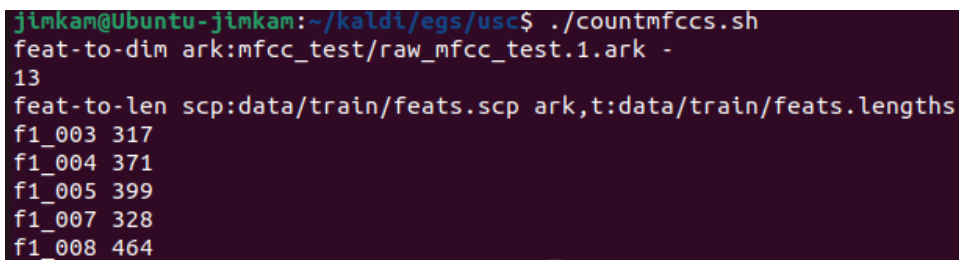
```
. ./path.sh # Needed for KALDI_ROOT
. ./cmd.sh

# find the dimensionality of feature vectors
feat-to-dim ark:mfcc_test/raw_mfcc_test.1.ark -

#Number of frames
feat-to-len scp:data/train/feats.scp ark,t:data/train/feats.lengths
cat data/train/feats.lengths | head -5
```

Ερώτημα 3

Τρέχουμε το παραπάνω script και έχουμε το εξής αποτέλεσμα:



```
jinkam@Ubuntu-jinkam:~/kaldi/egs/usc$ ./countmfccs.sh
feat-to-dim ark:mfcc_test/raw_mfcc_test.1.ark -
13
feat-to-len scp:data/train/feats.scp ark,t:data/train/feats.lengths
f1_003 317
f1_004 371
f1_005 399
f1_007 328
f1_008 464
```

Παρατηρούμε αρχικά ότι έχουμε 13 παράγοντες MFCCs για κάθε frames. Για τις 5 πρώτες προτάσεις του training set μπορούμε να δούμε στην συνέχεια του output τον αριθμό των ακουστικών frames που εξάγονται.

4.4 Εκπαίδευση ακουστικών μοντέλων και αποκωδικοποίηση προτάσεων

1. Με το παρακάτω script εκπαιδύουμε ένα monophone HMM-GMM model:

```

. ./path.sh # Needed for KALDI_ROOT
. ./cmd.sh

#=====4_4_1=====

steps/train_mono.sh --nj 4 --cmd "$train_cmd" \
data/train data/lang_test exp/mono0a || exit 1;

```

Όπως έχουμε αναφέρει και στο μέρος της θεωρητικής ανάλυσης ένα κρυφό μοντέλο Markov (HMM) μπορεί να μοντελοποιήσει την φύση της ομιλίας στον χρόνο και ένα μοντέλο GMM (Gaussian Mixture Model) μοντελοποιεί την κατανομή των ακουστικών χαρακτηριστικών (MFCCs) για κάθε φώνημα. Το μοντέλο που θα εκπαιδεύσουμε δεν θα χρησιμοποιεί πληροφορίες για το context κάθε φωνήματος και θα χειρίζεται έτσι κάθε φώνημα ξεχωριστά.

2. Για να δημιουργήσουμε τον γράφο HCLG του kaldι σύμφωνα με την γραμματική του προηγούμενου ερωτήματος χρησιμοποιούμε το παρακάτω bash script:

```

#!/bin/bash
source ./path.sh

# Rename G_train_unigram.fst to G.fst in order to
# run the mkgraph.sh which searches for an G.fst.

# unigram
cp data/lang_test/G_train_ug.fst data/lang_test/G.fst
# Make HCLG graph.
utils/mkgraph.sh --mono data/lang_test exp/mono0a exp/mono0a/graph_nosp_tgpr_ug

# bigram

cp data/lang_test/G_train_bg.fst data/lang_test/G.fst
# Make HCLG graph.
utils/mkgraph.sh --mono data/lang_test exp/mono0a exp/mono0a/graph_nosp_tgpr_bg

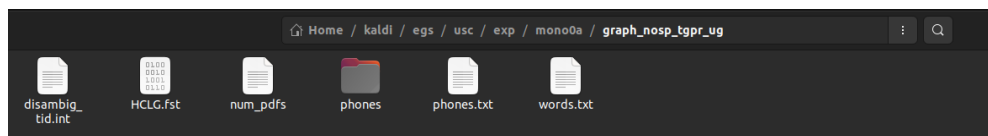
```

Σημειώνουμε ότι αλλάζουμε το G.fst κάθε φορά για να κάνουμε την πάνω διαδικασία και για unigram και για bigram μοντέλο. Δημιουργούμε δύο διαφορετικούς γράφους, τους οποίους αποθηκεύουμε σε δύο φακέλους με τα αντίστοιχα αναγνωριστικά για bigram και unigram model. Παρουσιάζουμε το αποτέλεσμα του script (μετά το run) και ενδεικτικά τα περιεχόμενα του φακέλου για το unigram model.

```

root@kali:~/kaldi/egs/usc/exp/mono0a$ ./A1.1.2.sh
WARNING: the --mono, --left-biphone and --right-biphone options are now deprecated and ignored.
tree-info exp/mono0a/tree
tree-info exp/mono0a/tree
fsttablecompose data/lang_test/_disambig.fst data/lang_test/g.fst
fstpushspecial
fstderminize --use-logtrue
fstinlnizeencoded
fststochastic data/lang_test/tmp/LG.fst
0.842346 0.0419453
[INFO]: LG not stochastic
fstcomposecontext --context-size=1 --central-position=0 --read-disambig-syms-data/lang_test/phones/disambig.int --write-disambig-syms-data/lang_test/tmp/disambig_labels_1_0.int data/lang_test/tmp/labels_1_0.1553 data/lang_test/tmp/LG.fst
fststochastic data/lang_test/tmp/CLG_1_0.fst
0.842346 0.0419453
[INFO]: CLG not stochastic
make-h-transducer --disambig-syms-out=exp/mono0a/graph_nosp_tgpr Ug/disambig_tid.int --transition-scale=1.0 data/lang_test/tmp/labels_1_0 exp/mono0a/tree exp/mono0a/final.mdl
fstnepslocal
fstsymbols exp/mono0a/graph_nosp_tgpr Ug/disambig_tid.int
fstinlnizeencoded
fsttablecompose exp/mono0a/graph_nosp_tgpr Ug/Ha.fst data/lang_test/tmp/CLG_1_0.fst
fstderminize --use-logtrue
fststochastic exp/mono0a/graph_nosp_tgpr Ug/HCLG.fst
0.842346 0.0419453
HCLG is not stochastic
add-self-loops --self-loop-scale=0.1 --reorder=true exp/mono0a/final.mdl exp/mono0a/graph_nosp_tgpr Ug/HCLG.fst
WARNING: the --mono, --left-biphone and --right-biphone options are now deprecated and ignored.
tree-info exp/mono0a/tree
tree-info exp/mono0a/tree
fsttablecompose data/lang_test/_disambig.fst data/lang_test/g.fst
fstpushspecial
fststochastic data/lang_test/tmp/LG.fst
0.842346 0.0419453
fstcomposecontext --context-size=1 --central-position=0 --read-disambig-syms-data/lang_test/phones/disambig.int --write-disambig-syms-data/lang_test/tmp/disambig_labels_1_0.int data/lang_test/tmp/labels_1_0.1556 data/lang_test/tmp/LG.fst
fststochastic data/lang_test/tmp/CLG_1_0.fst
0.842346 0.0419453
make-h-transducer --disambig-syms-out=exp/mono0a/graph_nosp_tgpr Bg/disambig_tid.int --transition-scale=1.0 data/lang_test/tmp/labels_1_0 exp/mono0a/tree exp/mono0a/final.mdl
fstnepslocal
fstsymbols exp/mono0a/graph_nosp_tgpr Bg/disambig_tid.int
fstinlnizeencoded
fstderminize --use-logtrue
fstnepslocal
fststochastic exp/mono0a/graph_nosp_tgpr Bg/HCLG.fst
0.842346 0.0419453
add-self-loops --self-loop-scale=0.1 --reorder=true exp/mono0a/final.mdl exp/mono0a/graph_nosp_tgpr Bg/HCLG.fst

```



3. Στην συνέχεια, θα χρησιμοποιήσουμε τον αλγόριθμο Viterbi για να αποκωδικοποιήσουμε τις προτάσεις των test και validation δεδομένων με το παρακάτω bash script:

```

source ./path.sh

echo "Running decode for testing set - unigram model"
steps/decode.sh exp/mono0a/graph_nosp_tgpr Ug data/test exp/mono0a/decode_test Ug

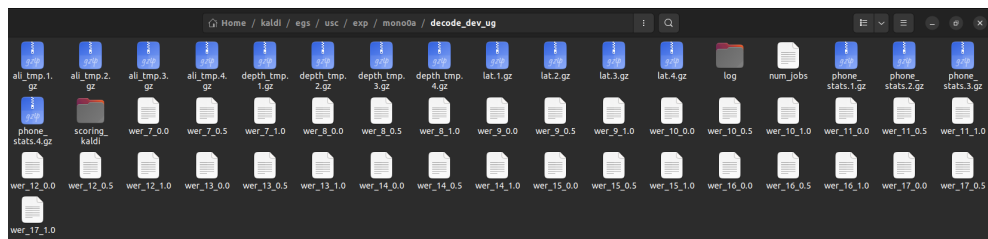
echo "Running decode for validation set - unigram model"
steps/decode.sh exp/mono0a/graph_nosp_tgpr Ug data/dev exp/mono0a/decode_dev Ug

echo "Running decode for testing set - bigram model"
steps/decode.sh exp/mono0a/graph_nosp_tgpr Bg data/test exp/mono0a/decode_test Bg

echo "Running decode for validation set - bigram model"
steps/decode.sh exp/mono0a/graph_nosp_tgpr Bg data/dev exp/mono0a/decode_dev Bg

```

Με το παραπάνω script φορτώνουμε κάθε φορά τον γράφο για την περίπτωση που ασχολούμαστε (unigram ή bigram), εξάγουμε τα mfccs features και τα χρησιμοποιούμε με τον αλγόριθμο Viterbi για να βρούμε τους πιο πιθανούς συνδυασμούς λέξεων για κάθε utterance. Τα αποτελέσματα αποθηκεύονται σε φακέλους, όπως αυτόν που φαίνεται παρακάτω (εδώ το παράδειγμα είναι ο φάκελος με τα δεδομένα από την αποκωδικοποίηση των dev δεδομένων του unigram model).



4. Στο βήμα αυτό θα παρουσιάσουμε τα αποτελέσματα της αποκωδικοποίησης με χρήση της μετρικής Phone Error Rate (PER):

$$PER = 100 \frac{insertions + substitutions + deletions}{\#phonemes}$$

Χρησιμοποιούμε το script που ακολουθεί στην συνέχεια. Το script αυτό θα υπολογίσει τα wer scores, τα οποία όμως στην περίπτωση μας συμπίπτουν με τα per scores, καθώς κωδικοποιούμε σε επίπεδο φωνήματος. Ψάχνουμε φυσικά το καλύτερο per, το οποίο σημαίνει το μικρότερο. Με το παρακάτω script για κάθε συνδυασμό set και model παρουσιάζουμε το βέλτιστο wer.

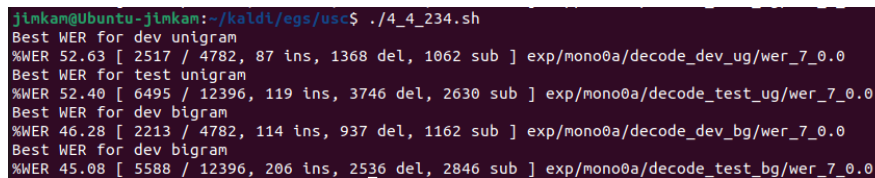
```
#!/bin/bash
source ./path.sh

# Print PER - dev unigram
echo "Best WER for dev unigram"
[ -d exp/mono0a/decode_dev_ug ] && grep WER exp/mono0a/decode_dev_ug/wer_*
| utils/best_wer.sh

echo "Best WER for test unigram"
# Print PER - test unigram
[ -d exp/mono0a/decode_test_ug ] && grep WER exp/mono0a/decode_test_ug/wer_*
| utils/best_wer.sh

echo "Best WER for dev bigram"
# Print PER - dev bigram
[ -d exp/mono0a/decode_dev_bg ] && grep WER exp/mono0a/decode_dev_bg/wer_*
| utils/best_wer.sh

echo "Best WER for dev bigram"
# Print PER.
[ -d exp/mono0a/decode_test_bg ] && grep WER exp/mono0a/decode_test_bg/wer_*
| utils/best_wer.sh
```



```
jinkam@Ubuntu-jinkam:~/kaldi/egs/usc$ ./4_4_234.sh
Best WER for dev unigram
%WER 52.63 [ 2517 / 4782, 87 ins, 1368 del, 1062 sub ] exp/mono0a/decode_dev_ug/wer_7_0.0
Best WER for test unigram
%WER 52.40 [ 6495 / 12396, 119 ins, 3746 del, 2630 sub ] exp/mono0a/decode_test_ug/wer_7_0.0
Best WER for dev bigram
%WER 46.28 [ 2213 / 4782, 114 ins, 937 del, 1162 sub ] exp/mono0a/decode_dev_bg/wer_7_0.0
Best WER for dev bigram
%WER 45.08 [ 5588 / 12396, 206 ins, 2536 del, 2846 sub ] exp/mono0a/decode_test_bg/wer_7_0.0
```

Παραπάνω φαίνονται τα αποτελέσματα του run του script. Παρατηρούμε πρώτα από όλα όπως είναι λογικό ότι και για τα δύο σύνολα έχουμε καλύτερα αποτελέσματα για τα bigram models. Το καλύτερο wer score πετυχαίνουμε στο dev set με το bigram model όπως φαίνεται και πάνω.

Για τις υπερπαραμέτρους της scoring διαδικασίας, η 1η είναι το word_ins.penalty, η οποία καθορίζει την ποινή της προσθήκης νέας λέξης κατά την διαδικασία της αποκωδικοποίησης. Η 2η είναι η lmwt (language model weight) και καθορίζεται από το μέγιστο και το ελάχιστο της, εδώ 7-17. Εκφράζει πόση επιρροή έχει το φωνητικό και πόση το γλωσσικό μοντέλο, με τις μικρότερες τιμές να δίνουν έμφαση στο φωνητικό και τις υψηλές στο γλωσσικό.

5. Στο τελευταίο βήμα κάνουμε alignment των φωνημάτων με το monophone model και με τα alignments εκπαιδεύουμε ένα triphone model, πραγματοποιούμε αποκωδικοποίηση και ακολουθούμε τα προηγούμενα βήματα.

```
#!/bin/bash

# Exit on error
set -e

# Number of parallel jobs
nj=4
cmd=run.pl

# Directories
data_train=data/train
data_dev=data/dev
lang_dir=data/lang_test
mono_dir=exp/mono0a
tri_dir=exp/tri1
graph_dir=$tri_dir/graph
decode_dir=$tri_dir/decode_dev

# Step 1: Align phonemes using monophone model
echo "Aligning phonemes with monophone model..."
steps/align_si.sh --nj $nj --cmd "$cmd" $data_train $lang_dir
$mono_dir ${mono_dir}_ali

# Step 2: Train a triphone model using the alignments
echo "Training triphone model..."
steps/train_deltas.sh --cmd "$cmd" 2000 10000 $data_train $lang_dir
${mono_dir}_ali $tri_dir

# Step 3: Create HCLG graph for decoding
echo "Building decoding graph..."
utils/mkgraph.sh $lang_dir $tri_dir $graph_dir

# Step 4: Decode test data using the triphone model
echo "Decoding with triphone model..."
steps/decode.sh --nj $nj --cmd "$cmd" $graph_dir $data_dev $decode_dir

# Step 5: Evaluate results
echo "Scoring the results..."
local/score.sh --cmd "$cmd" $data_dev $graph_dir $decode_dir

echo "All steps completed successfully!"
```

Πάνω φαίνεται το bash script που έχουμε χρησιμοποιήσει για αυτό το βήμα, στο οποίο μετά το alignment των φωνημάτων πραγματοποιούμε τα προηγούμενα βήματα εκπαίδευσης, δημιουργίας γράφου, αποκωδικοποίησης και evaluation, απλά αυτή την φορά για triphone model.

Ερώτημα 4

Ένα ακουστικό μοντέλο GMM-HMM χρησιμοποιείται σε προβλήματα αυτόματης αναγνώρισης φωνής (ASR). Τα μακροβιανά μοντέλα αναπαριστούν την ακολουθιακή φύση της ομιλίας και τα μίγματα Γκαουσιανών μοντελοποιούν την κατανομή των ακουστικών χαρακτηριστικών για κάθε κατάσταση μακροβιανού μοντέλου.

Ένα HMM αναπαριστά την ομιλία ως μια ακολουθία καταστάσεων. Κάθε κατάσταση αναπαριστά ένα τμήμα της ομιλίας και συνήθως αυτό σχετίζεται με ένα φώνημα ή το τμήμα ενός φωνήματος, αναλόγως αν ένα φώνημα αντιστοιχεί σε μία κατάσταση ή περισσότερες. Τα HMM έχουν πιθανότητες μετάβασης ανάμεσα στις καταστάσεις που καθορίζουν την πιθανότητα μετάβασης στο επόμενο στάδιο ή παραμονής σε αυτό.

Κάθε κατάσταση HMM συνδέεται με ένα GMM, το οποίο αναπαριστά την κατανομή των ακουστικών χαρακτηριστικών (mfccs). Κάθε GMM αποτελεί σταθμισμένο άθροισμα μίγματος Γκαουσιανών, κάθε μία από τις οποίες περιέχει φωνητική πληροφορία και ο συνδυασμός λειτουργεί ώστε να δωθούν πιθανότητες σε ακουστικά χαρακτηριστικά, για να καθορίσει τελικά το σύστημα ποιο φώνημα ακούστηκε. Φυσικά, όπως είναι λογικό, όσο περισσότερες Γκαουσιανές έχουμε, τόσο καλύτερη θα είναι και η αναγνώριση περίπλοκων φωνητικών παραλλαγών για τον ίδιο ήχο.

Η εκπαίδευση ενός τέτοιου μοντέλου περιλαμβάνει τα παρακάτω βήματα:

- Εξαγωγή χαρακτηριστικών

Πριν ξεκινήσει η εκπαίδευση εξάγονται τα mfccs από τον ήχο της ομιλίας.

- Εκπαίδευση για τα HMM

Αρχικά, κάθε κομμάτι ήχου (πχ φώνημα εδώ) αντιστοιχίζεται σε μια τυχαία κατάσταση HMM. Στην συνέχεια το εκπαιδεύουμε ώστε να γίνει σωστά η αντιστοίχιση.

- Εκπαίδευση GMM

Σε αυτό το βήμα για κάθε κατάσταση καθορίζονται μέσα από την εκπαίδευση οι παράμετροι των Γκαουσιανών και τα βάρη τους για κάθε κατάσταση.

Με την παραπάνω διαδικασία μπορούμε να εκπαιδεύσουμε και ένα monophone model, το οποίο θεωρεί ότι κάθε φώνημα είναι ανεξάρτητο από τα υπόλοιπα. Πριν την διαδικασία της εκπαίδευσης πρέπει να ακολουθήσουμε τις απαραίτητες διαδικασίες για να προετοιμάσουμε το γλωσσικό μοντέλο (π.χ. λεξικό, γραμματική), στην συνέχεια ακολουθούμε τα παραπάνω βήματα για εκπαίδευση και στο τέλος πρέπει να ακολουθήσει μία φάση decoding και scoring για να κριθεί η απόδοση του μοντέλου.

Ερώτημα 5

Για να βρεθεί η πιο πιθανή λέξη (φώνημα στη περίπτωση μας) δεδομένης μίας ακολουθίας ακουστικών χαρακτηριστικών, ορίζουμε ότι αυτή είναι η λέξη $\hat{W}$ που μεγιστοποιεί την a posteriori πιθανότητα $P(W|X)$, δεδομένου του διανύσματος χαρακτηριστικών X . Δηλαδή, εκφράζοντας σε μαθηματικό φορμαλισμό:

$$\hat{W} = \operatorname{argmax}_W P(W|X)$$

και από το θεώρημα Bayes:

$$\hat{W} = \operatorname{argmax}_W \frac{P(X|W)P(W)}{P(X)}$$

Σημειώνεται για τα παραπάνω ότι ο υπολογισμός της a posteriori πιθανότητας περιλαμβάνει 2 μέρη, πρώτον τον υπολογισμό της a priori πιθανότητας (γλωσσικό μοντέλο) και τον υπολογισμό της πιθανότητας το W να έχει παράξει το διάνυσμα χαρακτηριστικών X (ακουστικό μοντέλο). Σημειώνεται ότι στον πάνω τύπο, εφόσον το $P(X)$ είναι σταθερό για όλα τα φωνήματα μπορεί να παραβλεφθεί από την βελτιστοποίηση.

Ερώτημα 6

Ο HCLG graph στο kaldι αναπαριστά όλες τις πιθανές ακολουθίες λέξεων που μπορεί να αναγνωρίσει ένα σύστημα αυτόματης αναγνώρισης φωνής. Αποτελείται από τα εξής επίπεδα:

- H (HMM)

Κωδικοποιεί τις μεταβάσεις μεταξύ καταστάσεων για κάθε φώνημα και προκύπτει από τα εκπαιδευμένα ακουστικά μοντέλα.

- C (Context Dependency Model)

Μοντελοποιεί τις σχέσεις σε ένα triphone model με το προηγούμενο και το επόμενο φώνημα και ουσιαστικά καθορίζει το ηχητικό αποτέλεσμα των φωνημάτων και βάσει των γειτόνικών φωνημάτων.

- L (Lexicon)

Ουσιαστικά αντιστοιχεί λέξεις σε φωνήματα, αποθηκεύει τις σχέσεις τους και περιλαμβάνει διαφορετικές προφορές φωνημάτων.

- G (Grammar)

Είναι το γλωσσικό μοντέλο, καθορίζει τις πιθανότητες λέξεων και μοντελοποιεί τις πιθανές μεταβάσεις μεταξύ λέξεων.

Με σύνθεση των παραπάνω στοιχείων προκύπτει ο HCLG γράφος που μπορεί να πραγματοποιεί την αποκωδικοποίηση, στην οποία μπορεί ταυτόχρονα να υπολογίζει τις πιθανότητες εμφάνισης λέξεων και να αναλύει τα ακουστικά χαρακτηριστικά.

Βιβλιογραφία

- Theory and Applications of Digital Speech Processing, Lawrence R. Rabiner and Ronald W. Schafer (Pearson, 2011).
- <https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html>
- https://maelfabien.github.io/machinelearning/speech_reco_1/#context-dependent-phone-models